# Asignación de horarios universitarios: búsqueda no informada y A* sobre un problema de satisfacción de restricciones

Juan Sebastian Pérez - 2459371, Victor Murillo - 2459569

Universidad del Valle — Inteligencia Artificial

## Índice

**1. Descripción del problema** **3**

**2. Formalización del problema** **3**
2.1. Variables y dominios . . . 3
2.2. Restricciones duras . . . 3
2.3. Restricciones blandas . . . 3
2.4. Función de costo . . . 4

**3. Representación del estado y espacio de búsqueda** **4**
3.1. Representación del estado . . . 4
3.2. Espacio de búsqueda . . . 4

**4. Heurística de A*** **5**
4.1. Definición . . . 5
4.2. Admisibilidad . . . 5
4.3. Consistencia . . . 5
4.4. Verificación en el programa . . . 6
4.5. Limitación . . . 6
4.6. Ejemplo: `datos_penalizacion/` . . . 6

**5. Pruebas y resultados** **7**
5.1. Conjuntos de datos . . . 7
5.2. Verificación de la implementación . . . 7
5.3. Resultados comparativos . . . 7
5.4. Interfaz gráfica . . . 7

**6. Análisis** **8**
6.1. Por qué BFS e IDS no son óptimos . . . 8
6.2. Por qué A* reduce nodos solo en `datos_penalizacion/` . . . 8
6.3. Limitaciones . . . 8

**7. Conclusiones** **8**

**8. Anexo: capturas de la interfaz** **9**

# 1. Descripción del problema

Una universidad debe construir el horario académico de un semestre. Cada clase necesita una franja horaria y un salón, de modo que ninguna de las restricciones del sistema se viole. El problema pertenece a la familia de los Problemas de Satisfacción de Restricciones (CSP), y en este trabajo se resuelve como un problema de búsqueda en espacios de estados.

El objetivo del proyecto es implementar y comparar cuatro estrategias: búsqueda en amplitud (BFS), búsqueda de costo uniforme (UCS), búsqueda en profundidad iterativa (IDS) y A*. Las tres primeras son no informadas y A* es informada, porque usa una heurística para guiar la búsqueda.

Además de encontrar un horario válido, el sistema busca uno de bajo costo. Por eso el problema tiene restricciones duras, que nunca pueden violarse, y restricciones blandas, que son deseables pero se penalizan cuando no se cumplen.

# 2. Formalización del problema

## 2.1. Variables y dominios

Cada clase que debe programarse es una variable. Sea $C = \{c_1, \ldots, c_N\}$ el conjunto de clases, donde $N = |C|$. El dominio de cada variable es el conjunto de combinaciones posibles de franja y salón:

$$D_i = F \times S,$$

donde $F$ es el conjunto de franjas horarias y $S$ el conjunto de salones. Cada franja tiene un día y una hora, y su posición dentro del día determina qué franjas son consecutivas.

## 2.2. Restricciones duras

Las siguientes restricciones no pueden violarse. Una asignación que las incumpla se descarta y nunca forma parte de un estado:

1. Un salón no puede tener dos clases en la misma franja.
2. Un profesor no puede dictar dos clases en la misma franja.
3. Un grupo de estudiantes no puede tener dos clases en la misma franja.
4. Cada clase se asigna exactamente una vez.

## 2.3. Restricciones blandas

Estas restricciones son deseables. Si se violan, el horario sigue siendo válido, pero su costo aumenta:

1. **Disponibilidad:** un profesor no debería dictar en una franja que marcó como no disponible. Cada violación suma $P_{\text{disp}} = 5$.
2. **Consecutivas:** un grupo no debería tener más de $M = 3$ clases seguidas en un mismo día. Cada clase de exceso suma $P_{\text{cons}} = 3$.

### 2.4. Función de costo

Cada asignación de una clase tiene un costo de paso igual a una unidad base más las penalizaciones blandas que genera:

$$c(a) = 1 + \pi(a).$$

El costo de un horario completo $s$ con $N$ clases es

$$\text{costo}(s) = N + \sum_{a \in s} \pi(a),$$

donde la suma de penalizaciones se llama *penalización* del horario. La solución óptima es la de menor costo entre todos los horarios válidos.

## 3. Representación del estado y espacio de búsqueda

### 3.1. Representación del estado

Las clases se asignan siempre en el orden en que aparecen en `cursos.txt`. Un estado es la lista de asignaciones de las primeras $k$ clases. El estado inicial tiene $k = 0$ y la meta tiene $k = N$.

Como el orden de asignación es fijo, cada estado se alcanza por un único camino. Por esta razón no es necesario mantener una lista de estados visitados.

### 3.2. Espacio de búsqueda

El espacio de búsqueda es un árbol de profundidad $N$. La raíz es el estado vacío, cada nivel decide la asignación de una clase y las hojas del nivel $N$ son horarios completos. El factor de ramificación en cada nivel es, como máximo,

$$b = |F| \cdot |S|,$$

aunque las restricciones duras reducen ese número en la práctica. Sin poda, el árbol tendría del orden de $b^N$ nodos, lo que explica por qué el límite de nodos `max_nodos` es necesario incluso con datos pequeños.

| Parámetro | Valor en `datos/` |
|---|---|
| Clases $N$ | 4 |
| Franjas $\lvert F \rvert$ | 4 |
| Salones $\lvert S \rvert$ | 2 |
| Factor máximo $b = \lvert F \rvert \cdot \lvert S \rvert$ | 8 |

Cuadro 1: Tamaño del espacio de búsqueda con el conjunto de datos de ejemplo.

# 4. Heurística de A*

## 4.1. Definición

A* ordena la frontera por $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo real acumulado desde el estado inicial hasta $n$ y $h(n)$ es una estimación del costo que falta para llegar a una meta.

Para definir $h$, sea $C = \{c_1, \ldots, c_N\}$ el conjunto de clases en el orden de asignación. Para cada clase $c_i$ se define su penalización mínima posible:

$$m_i = \begin{cases} 0 & \text{si el profesor de } c_i \text{ tiene al menos una franja disponible,} \\ P_{\text{disp}} = 5 & \text{si todas sus franjas están marcadas como no disponibles.} \end{cases}$$

Si un estado $n$ tiene $k$ clases asignadas, las clases restantes son $c_{k+1}, \ldots, c_N$. La heurística se define como

$$h(n) = \sum_{i=k+1}^{N} \big(1 + m_i\big).$$

Como $m_i$ depende solo de la clase y no del estado, la suma se puede precalcular una sola vez con una tabla de sufijos $H_k = \sum_{i=k+1}^{N}(1 + m_i)$, con $H_N = 0$. Entonces $h(n) = H_k$ se obtiene en tiempo constante.

## 4.2. Admisibilidad

**Proposición.** Para todo estado $n$, $h(n) \leq h^*(n)$, donde $h^*(n)$ es el costo real mínimo para completar $n$ hasta una meta.

**Demostración.** Sea $s$ un horario completo que extiende a $n$ y que es óptimo para $n$. Las clases restantes $c_{k+1}, \ldots, c_N$ tienen asignaciones $a_{k+1}, \ldots, a_N$ en $s$. El costo de cada una es $1 + \pi(a_i)$, donde $\pi(a_i)$ es la suma de sus penalizaciones blandas.

La penalización de disponibilidad de $a_i$ vale 5 si la franja asignada está en la lista de no disponibles del profesor de $c_i$, y 0 en otro caso. Por definición, $m_i$ es el mínimo de esa penalización sobre todas las franjas posibles para $c_i$. Por lo tanto, $\pi_{\text{disp}}(a_i) \geq m_i$.

La penalización de consecutivas nunca es negativa, así que $\pi(a_i) \geq \pi_{\text{disp}}(a_i) \geq m_i$. Luego

$$h^*(n) = \sum_{i=k+1}^{N} \big(1 + \pi(a_i)\big) \geq \sum_{i=k+1}^{N} \big(1 + m_i\big) = h(n).$$

■

Nótese que la demostración no depende de las restricciones duras: el mínimo se toma sobre todas las franjas, así que la cota sigue valiendo aunque las duras eliminen algunas combinaciones.

## 4.3. Consistencia

**Proposición.** Para todo nodo $n$ y todo sucesor $n'$ generado con costo de paso $c(n, n')$,

$$h(n) \leq c(n, n') + h(n').$$

**Demostración.** Sea $c_{k+1}$ la clase que se asigna al pasar de $n$ a $n'$. Entonces $h(n) - h(n') = 1 + m_{k+1}$. El costo del paso es $c(n, n') = 1 + \pi(a_{k+1}) \geq 1 + m_{k+1}$, por la misma argumentación de la admisibilidad aplicada a una sola clase. Por lo tanto

$$h(n) - h(n') = 1 + m_{k+1} \leq 1 + \pi(a_{k+1}) = c(n, n').$$

■

Como la heurística es consistente, $f$ no disminuye a lo largo de ningún camino, y A* devuelve la solución óptima al extraer la primera meta de la frontera. Además, cada nodo se expande como máximo una vez, aunque en este proyecto el árbol no tiene estados repetidos.

## 4.4. Verificación en el programa

Las dos propiedades se comprueban en `pruebas.py`. La admisibilidad se verifica en el estado inicial, comparando $h$(inicial) con el costo óptimo encontrado por costo uniforme. La consistencia se verifica en todas las transiciones del árbol de `datos/`, que son 1304 pares padre-hijo, sin encontrar violaciones.

## 4.5. Limitación

La heurística no estima la penalización de consecutivas. Esa penalización depende de cómo se combinan las clases de un mismo grupo en un día, y no existe una cota inferior simple que no dependa del resto de las asignaciones. Por eso $m_i$ se calcula solo con la disponibilidad del profesor. La consecuencia es que, en los datos de `datos/`, todos los profesores tienen franjas libres, $m_i = 0$ para todas las clases y $h$ coincide con la cantidad de clases restantes. En ese caso A* explora los mismos nodos que costo uniforme.

## 4.6. Ejemplo: `datos_penalizacion/`

En esta carpeta el profesor Prof_B (que dicta la clase C3) tiene todas sus franjas marcadas como no disponibles. Entonces $m_3 = 5$ y el resto de los mínimos son 0. La heurística inicial vale

$$h(\text{inicial}) = \underbrace{(1+0) + (1+0) + (1+5) + (1+0)}_{N=4 \text{ clases}} = 9,$$

que coincide con el costo óptimo. La tabla 2 muestra el efecto sobre la cantidad de nodos explorados.

| Carpeta | Algoritmo | Costo óptimo | Nodos explorados |
|---|---|---|---|
| `datos/` | UCS | 4 | 152 |
| `datos/` | A* | 4 | 152 |
| `datos_penalizacion/` | UCS | 9 | 178 |
| `datos_penalizacion/` | A* | 9 | 152 |

Cuadro 2: Nodos explorados por costo uniforme y A* en los dos conjuntos con solución.

La reducción de 178 a 152 nodos se debe a que A* ya suma la penalización de C3 desde la raíz. Así, los nodos cuyo costo mínimo supera 9 se descartan antes que en costo uniforme, que solo conoce la penalización al asignar esa clase.

# 5. Pruebas y resultados

## 5.1. Conjuntos de datos

Se usan tres conjuntos de datos, cada uno con un propósito distinto:

- `datos/`: horario con solución y heurística que no aporta información, porque todos los profesores tienen al menos una franja disponible.
- `datos_penalizacion/`: igual que el anterior, pero Prof_B tiene todas sus franjas marcadas como no disponibles. Sirve para mostrar el efecto de la heurística.
- `datos_sin_solucion/`: caso sin horario válido, para comprobar que los algoritmos terminan y reportan la falta de solución.

## 5.2. Verificación de la implementación

El archivo `pruebas.py` ejecuta pruebas automáticas sobre la lectura de datos, la detección de errores de formato, las restricciones duras y blandas, la cola FIFO y la cola de prioridad, las cuatro búsquedas, la admisibilidad y consistencia de la heurística, el límite de nodos y la tabla de estadísticas. Las pruebas usan los conjuntos `datos/` y `datos_sin_solucion/`. Los resultados de `datos_penalizacion/` se obtuvieron con una corrida independiente y se reportan en la tabla 3.

## 5.3. Resultados comparativos

La tabla 3 resume el costo de la solución y los nodos explorados por cada algoritmo.

| Carpeta | Algoritmo | Costo | Penalización | Nodos explorados |
|---|---|---|---|---|
| `datos/` | UCS | 4 | 0 | 152 |
| `datos/` | A* | 4 | 0 | 152 |
| `datos/` | BFS | 9 | 5 | 298 |
| `datos/` | IDS | 9 | 5 | 369 |
| `datos_penalizacion/` | UCS | 9 | 5 | 178 |
| `datos_penalizacion/` | A* | 9 | 5 | 152 |
| `datos_penalizacion/` | BFS | 14 | 10 | 298 |
| `datos_penalizacion/` | IDS | 14 | 10 | 369 |
| `datos_sin_solucion/` | todos | sin solución | | |

Cuadro 3: Costo, penalización y nodos explorados por algoritmo.

Las figuras 1 y 2 muestran esta comparación en la interfaz gráfica.

Los tiempos de ejecución varían entre corridas y se miden con `time.perf_counter()`, por lo que no se incluyen en la tabla.

## 5.4. Interfaz gráfica

La interfaz muestra el horario en una grilla con días en columnas y franjas en filas, y una tabla de estadísticas con una fila por algoritmo ejecutado. Para `datos_sin_solucion/` la

grilla queda vacía y se indica “No existe solución”. Las capturas de cada caso se incluyen en el anexo.

# 6. Análisis

## 6.1. Por qué BFS e IDS no son óptimos

BFS e IDS no consideran el costo de los pasos: BFS recorre el árbol por niveles y devuelve el primer horario completo que encuentra, e IDS hace lo mismo con límites de profundidad crecientes. Como las penalizaciones no influyen en el orden de exploración, el horario hallado puede tener una penalización alta. En `datos/` el costo sube de 4 a 9, y en `datos_penalizacion/` de 9 a 14.

## 6.2. Por qué A* reduce nodos solo en `datos_penalizacion/`

Con `datos/`, la heurística vale lo mismo que la cantidad de clases restantes. Entonces $f$ es igual para todos los nodos de un mismo nivel, y A* desempata igual que costo uniforme, explorando los mismos 152 nodos.

Con `datos_penalizacion/`, la heurística incluye la penalización mínima de Prof_B desde la raíz. Así los caminos que costarían más de 9 se descartan antes, y A* explora 152 nodos frente a 178 de costo uniforme.

## 6.3. Limitaciones

La heurística no modela las penalizaciones de consecutivas, por lo que en instancias donde esa restricción es la que más pesa, la reducción de nodos podría ser menor. Este caso no se midió en este trabajo. Además, el número de nodos podría depender del orden de las clases en `cursos.txt`, porque la asignación sigue ese orden fijo. Tampoco se midió esa dependencia.

# 7. Conclusiones

Se implementaron cuatro estrategias de búsqueda para el problema de asignación de horarios universitarios: búsqueda en amplitud, costo uniforme, profundidad iterativa y A*. Todas las colas (la fila de BFS y la cola de prioridad de UCS y A*) se implementaron desde cero, sin usar estructuras de la biblioteca estándar; solo se usan `time` para medir tiempos y `tkinter` para la interfaz.

Los resultados confirman tres ideas. Primero, costo uniforme y A* encuentran siempre el horario óptimo, mientras que BFS e IDS entregan soluciones válidas pero más costosas, porque no consideran las penalizaciones blandas. Segundo, la heurística de A* es admisible y consistente, y en `datos_penalizacion/` reduce los nodos explorados de 178 a 152 sin cambiar el resultado. Tercero, la reducción depende de los datos: con `datos/` la heurística no aporta información y A* se comporta igual que costo uniforme.

La principal limitación es que la heurística no estima las penalizaciones de consecutivas. Una mejora natural sería calcular una cota para esa restricción, aunque sería más difícil de demostrar admisible. Otra línea de trabajo sería reemplazar el árbol de asignaciones por una búsqueda con poda de estados, para evitar recorrer combinaciones que ya violan restricciones duras.

# 8. Anexo: capturas de la interfaz

Las capturas de la interfaz gráfica se incluyen en las figuras siguientes.

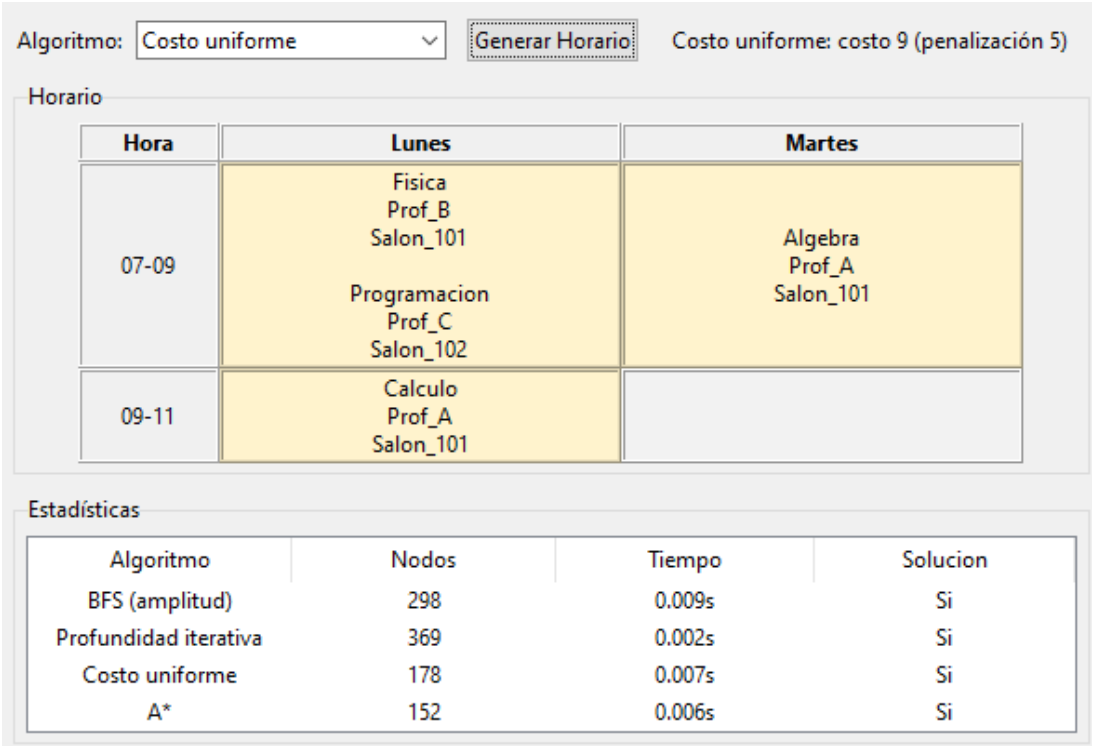



Figura 1: Costo uniforme sobre `datos_penalizacion/`: costo 9 y 178 nodos explorados.

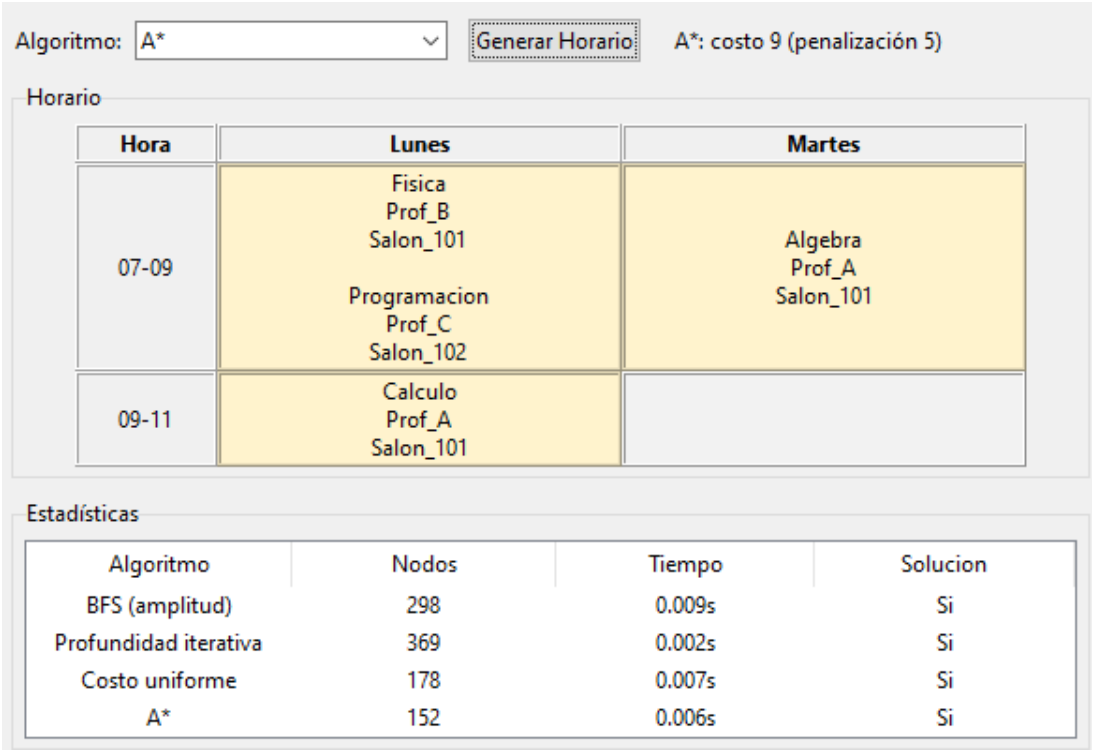



Figura 2: A* sobre `datos_penalizacion/`: costo 9 y 152 nodos explorados.

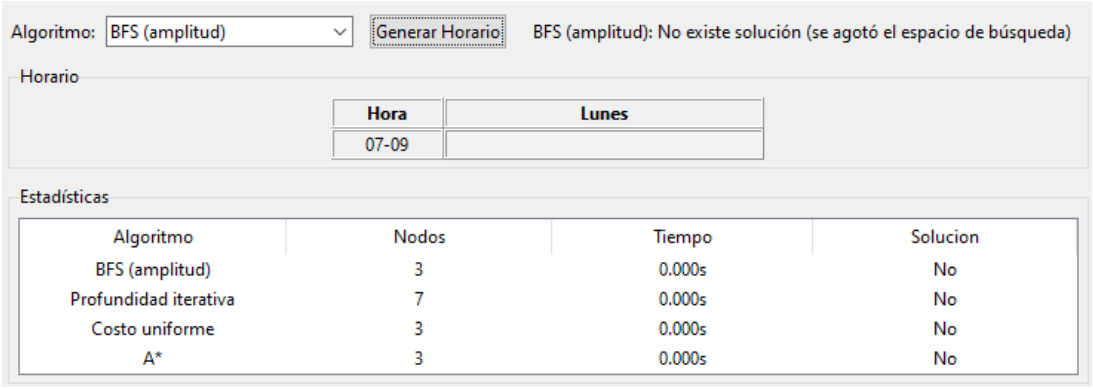



Figura 3: Caso `datos_sin_solucion/`: grilla vacía y mensaje "No existe solución".




Figura 4: Error de lectura al cargar un archivo en el campo equivocado.